CVAI Bill Analysis

Master guide — pipeline, preprocessing, architecture, parameters, and every question answered

The project in five sentences

1. You give it a photo of a shop bill.
2. It cleans the photo and reads the text with a ready-made AI reader called PaddleOCR.
3. It organises the text into bill lines and picks out the shop name, date, items, taxes and total.
4. It does the bill's maths itself: do the items plus taxes really equal the total?
5. If the maths works it says VALID; if one small, well-supported fix makes it work it says REPAIRED; otherwise it says CHECK_REQUIRED so a person checks it.

The key idea: a bill checks itself. If the numbers we read add up exactly, we almost certainly read them right. That is why a VALID result can be trusted — on 50 receipts the system had never seen, it was right 29 times out of 30.

Contents

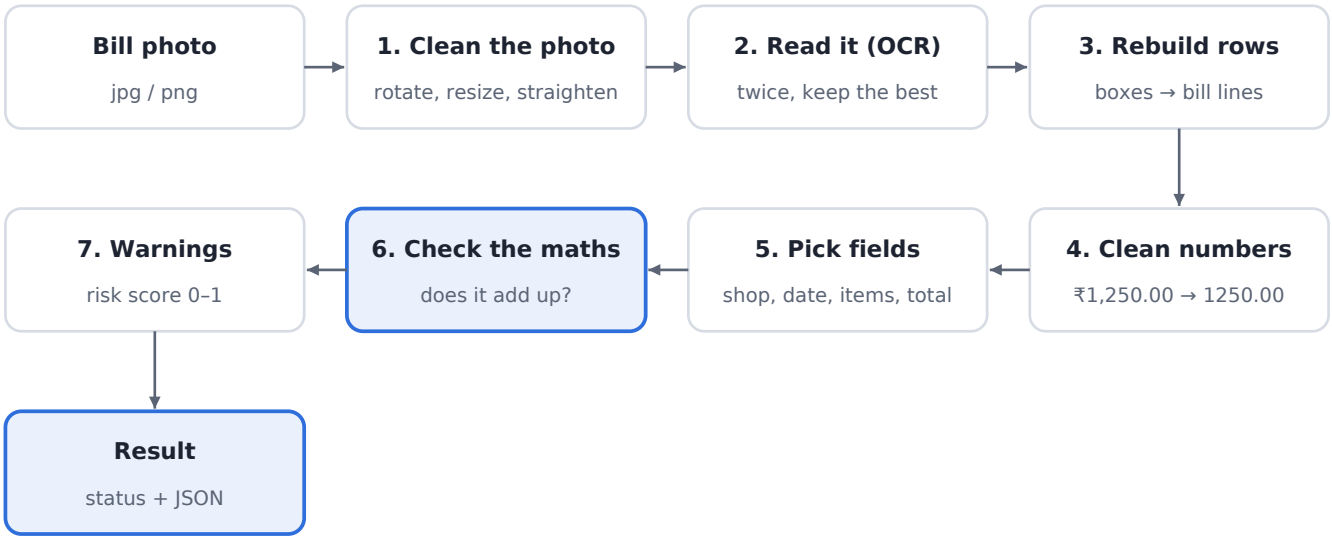
Part	What it covers
0. Basics	Pixels, OCR, neural network, library — the words used everywhere else
1. Pipeline	Input → 7 steps → output, with one example bill followed all the way
2. Preprocessing	Every photo-cleaning step: what, why, how, with pictures
3. Technical architecture	Table of every component: meaning, role, why chosen, alternatives, limits — then deep dives
4. Parameters	Every setting: value, what it controls, what happens if raised or lowered, how to tune it
5. All your questions	Every doubt asked so far, answered fully
6. Results and viva	Numbers that count, short answers, things to confirm

Part 0 — Basic words you need first

Word	Meaning
Pixel	A photo is a grid of tiny dots called pixels. A 2000 × 1000 photo has 2 million. A colour pixel stores 3 numbers from 0 to 255 (red, green, blue). A grey pixel stores 1 number: brightness, 0 = black, 255 = white.
Computer vision	Getting meaning out of those grids of numbers — here: finding and reading the text on a bill.
OCR	Optical Character Recognition: the task of finding text in an image and turning it into typed characters.
PaddleOCR	The specific OCR tool this project uses (free, from Baidu). OCR is the job; PaddleOCR is the worker that does it.
Neural network	A program that learned a skill from millions of examples instead of following hand-written rules. The only neural networks in this project are inside PaddleOCR.
Pretrained	Already trained by someone else. The project uses PaddleOCR as-is and trains nothing.
Rule-based	Hand-written if-this-then-that logic. Every step after reading is rule-based, so every decision can be explained.
Library	Ready-made code you call instead of writing it yourself (Pillow, OpenCV, RapidFuzz…).
Bounding box	The rectangle OCR draws around one line of text it found.
Confidence	A number from 0 to 1 saying how sure a step is. 0.97 = very sure.
JSON	Text that stores data as named fields, e.g. "total": 378.00. Easy for people and programs.
Parameter	A fixed setting in the code, chosen by a person (e.g. "allow 0.1% difference").

Part 1 — The pipeline (input → … → output)

In one line: photo → clean it → read it twice → rebuild the bill's lines → turn text into exact numbers → pick out each field → check the maths → raise warnings → JSON result. Steps 1–4 turn pixels into numbers. Steps 5–7 decide what the numbers mean and whether they can be trusted.



Example bill used throughout (made up for explanation)

```
SPICE HUT RESTAURANT
Date: 12/01/2024        Bill No: 457
Item           Qty   Rate     Amount
Paneer Tikka     2   120.00   240.00
Butter Naan      4    30.00   120.00
Sub Total                      360.00
CGST @ 2.5%                      9.00
SGST @ 2.5%                      9.00
Grand Total                    378.00
Cash 400.00          Change 22.00
```

Photographed with the phone held upright; the bill is tilted about 3°.

Input (app.py, scripts/run_stage12.py)

Takes in: a file you upload, or a file in data/desktop/images.
Gives out: the photo, ready for step 1.

How:

- Accepted types: .jpg .jpeg .png .bmp .webp .tif .tiff.
- One bill per image. Bad photos are not rejected — they are only tagged (step 1).

Example bill: bill.jpg, 3000 × 4000 pixels, stored sideways by the phone with an EXIF rotation note.

Why this step exists: Anything a phone or scanner produces should work.

Step 1 — Preprocess (clean the photo) (src/preprocess.py)

Takes in: the photo file.
Gives out: "orig" (upright, sized, straight) + "enh" (orig with better contrast, less grain) + tilt angle + quality numbers.

How:

- Open with Pillow and apply the EXIF rotation → upright.
- Resize so the long side is between 1000 and 2000 pixels.
- Measure the tilt with Otsu + minAreaRect; rotate straight if 0.5°–15°.
- Measure blur and brightness; tag LOW_QUALITY if either is below 60.
- Make enh: CLAHE contrast boost on lightness only, then light denoising.
- Full detail in Part 2.

Example bill: Upright, 1500 × 2000 pixels, rotated by 3°, tag OK; two versions.

Why this step exists: OCR reads upright, straight, clear text far better.

Step 2 — Two-pass OCR (read it twice) (src/ocr.py)

Takes in: orig and enh.
Gives out: a list of text lines: box (4 corners), text, confidence.

How:

- PaddleOCR reads orig: detector finds each text line, direction checker flips upside-down ones, recogniser reads the characters.
- PaddleOCR reads enh the same way.
- Merge by IoU (box overlap): overlap > 0.5 = same line → keep the more confident reading; enh-only boxes with overlap < 0.3 to everything kept → added; in between → dropped as duplicates.

Example bill: orig read "Grand Tota1 378.0O" (0.83); enh read "Grand Total 378.00" (0.97) → enh kept.

Why this step exists: Each version reads some lines better; merging keeps the best of both.

Step 3 — Rebuild rows (src/rows.py)

Takes in: loose OCR boxes.
Gives out: rows (bill lines) split into cells, each with labels and amounts.

How:

- Sort boxes top to bottom.
- A box joins the current row if it overlaps the row's height by at least half (ROW_OVERLAP 0.5).
- Inside a row, a gap wider than 2.5 average characters starts a new cell (column).
- Label each row with fuzzy keyword matching (TOTAL, CGST, CASH…), score ≥ 85.

Example bill: Row 4: "Paneer Tikka | 2 | 120.00 | 240.00". Row 9: "Grand Total | 378.00" → label GRAND_TOTAL.

Why this step exists: A bill is a table; rows tell you which price belongs to which item.

Step 4 — Normalize (text → exact values) (src/normalize.py)

Takes in: the text of each row.
Gives out: Decimal amounts with a "strong/weak" mark; dates as YYYY-MM-DD.

How:

- Strip currency signs (₹, Rs, INR, $) and edge symbols.
- Reject %, times (12:30), date shapes, and integers over 6 digits (phone, HSN codes).
- Fix O→0, I/l→1, S→5, B→8 inside numbers that already have 2+ digits.
- Commas: "1,250" = thousands; "12,50" = decimal comma.
- Read 5 date formats; day-first by default (Indian style).

Example bill: "378.0O" → 378.00; "12/01/2024" → 2024-01-12.

Why this step exists: Step 6 needs exact numbers; Decimal makes the maths exact.

Step 5 — Extract fields (src/extract.py)

Takes in: labelled rows with values.
Gives out: merchant, date, GSTIN, bill no., items, taxes, subtotal, discount, round-off, payments, total — each with confidence, evidence rows and runner-ups.

How:

- Find the header row (Item / Qty / Rate…) and where totals start (Sub Total…).
- Items = rows in between; qty × rate = amount checked.
- Taxes from ONE source only (rate lines, tax lines, or a GST table), never added twice.
- Every total candidate is scored; the top 3 are kept.

Example bill: Grand Total 378: label +3, near bottom +1, = items + tax +3, = cash − change +3 → wins. Sub Total 360: label −2. Cash 400: −3.

Why this step exists: Bills have many "total" lines; scoring several clues beats trusting one word.

Step 6 — Validate (check the maths) (src/validate.py)

Takes in: the fields.
Gives out: status, tax mode, difference, repairs, reasons, witnesses, confidence.

How:

- Tax on top: 360 + 9 + 9 − 378 = 0. Tax inside: 360 − 378 = −18. Smaller wins: tax on top.
- Allowed difference = max(0.05, 0.1% of total) = 0.38 → 0 is inside → balances.
- Witnesses (independent proof of the total): on a GRAND TOTAL line; cash − change = 378.
- If it had not balanced: try repairs, accept only a unique fix with a witness.

Example bill: VALID, no repairs.

Why this step exists: The heart of the project: correct arithmetic proves correct reading.

Step 7 — Anomaly flags (src/anomaly.py)

Takes in: fields + validation.
Gives out: flags and a risk score 0–1.

How:

- 7 rules, each with a weight: does_not_balance 0.5; tax_rate_mismatch, cgst_ne_sgst, total_not_supported, round_off_too_large 0.3 each; repaired_value 0.25; duplicate_item_rows 0.15.
- Risk = sum of raised weights, capped at 1.0.

Example bill: No flags → risk 0.

Why this step exists: A quick, explained reason for a person to look at a bill.

Output (app.py)

Takes in: everything above.
Gives out: JSON with fields, validation and anomaly; web page display.

How:

- The app shows the status, the photo with OCR boxes (green = original pass, blue = enhanced pass), the fields, the maths check, repairs, reasons and flags.
- "Download JSON result" saves bill_result.json.

Example bill: {"merchant": "SPICE HUT RESTAURANT", "date": "2024-01-12", "total": 378.00, "status": "VALID", "risk": 0}

Why this step exists: Readable by people and usable by other programs.

The four possible results

Status	Meaning	When
VALID	The numbers add up exactly as read	Balances, no fixes
REPAIRED	Adds up after one proven fix	Exactly one fix works and a witness supports the total; fix logged (old, new, reason)
CHECK_REQUIRED	Not proven — a person must check	Does not add up, or the fix had no proof
INSUFFICIENT_EVIDENCE	Too little was read to check	No total, or no items / taxes / subtotal

When the maths does NOT work — real fresh bill_19

OCR read the total as **24.04**; the items add up to **2149.00**. The gap (~2125) is far above the allowed 2.15, and nothing on the bill supports changing the total. So the system does not guess: **CHECK_REQUIRED**, with 2149.00 shown as a hint ("total_derived"). The true total was 2149.

Two ways the pipeline runs

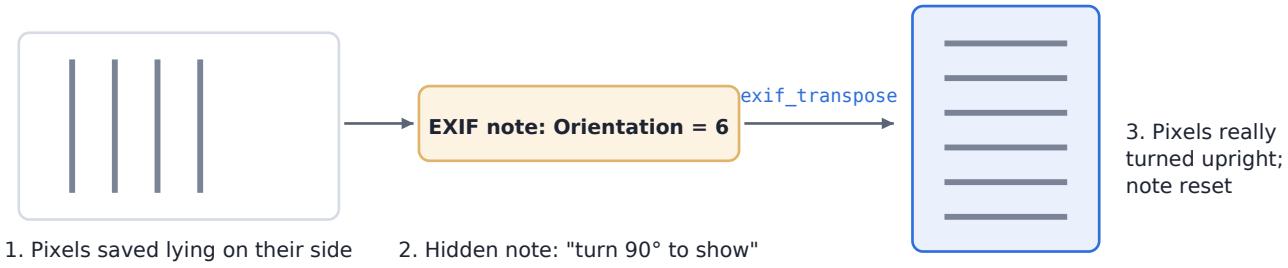
- **Web app (live):** steps hand results to each other in memory — `preprocess.run` → `ocr.two_pass` → `rows.build_rows` → `extract.extract` → `validate.validate` → `anomaly.score`. Nothing is saved.
- **Test scripts:** each step saves its result in outputs/stages/<bill>/ (01_orig.jpg … 06_anomaly.json) and the next script reads it. OCR takes ~9 s per bill, so saving it lets the fast rule steps be re-run in seconds.

Part 2 — Preprocessing (Step 1 in detail)

Yes, there is preprocessing. Five operations, in this order, all in `src/preprocess.py`: (1) load + EXIF rotation, (2) resize, (3) deskew, (4) quality check, (5) enhancement (CLAHE + denoise). Output: two images, **orig** and **enh**. Rule: the photo the OCR reads is never turned into pure black and white.

2.1 Load and EXIF rotation

What: EXIF is hidden information inside every phone photo (date, phone model, location) including an **Orientation** number. Phone sensors capture in one fixed direction, so instead of rotating millions of pixels the phone saves the picture as captured and writes a note, e.g. "Orientation = 6" = turn 90° clockwise to show.



Why: gallery apps obey the note, so the photo looks fine to you — but Python does not, so the OCR would receive a sideways bill and read almost nothing.

How: `Image.open` (Pillow opens the file) → `ImageOps.exif_transpose` (rotate the pixels to match the note, then reset the note to 1) → `.convert("RGB")` (3-colour image) → `np.asarray` (grid of numbers) → `cv2.cvtColor` RGB2BGR (OpenCV's colour order).



Without converting, OpenCV would read the first number (200) as BLUE, so a red stamp would become blue. Colours would be wrong and contrast steps that work on colour would misbehave.

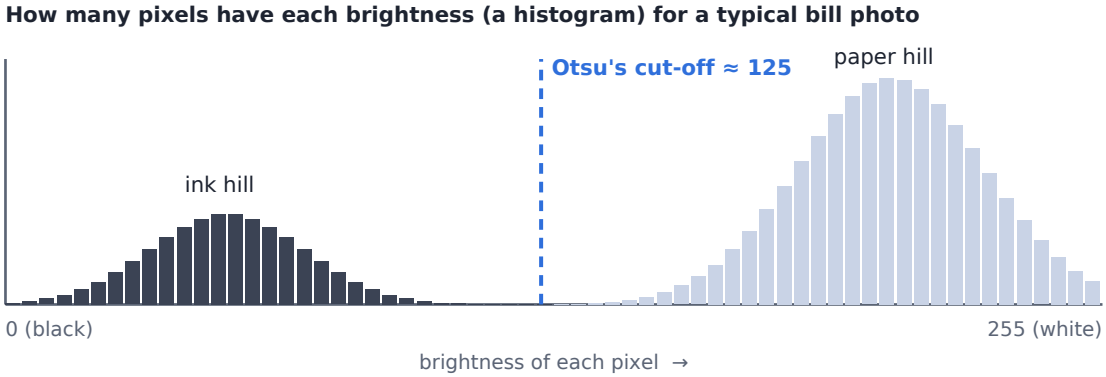
2.2 Resize

Situation	Action	Method and why
Long side > 2000 px (most phone photos: 3000–4000)	Shrink to 2000	INTER_AREA: averages groups of pixels — the cleanest way to shrink, no jagged edges. 2000 px still keeps receipt text sharp; OCR time grows with pixel count.
Long side < 1000 px (small screenshots)	Enlarge to 1200	INTER_CUBIC: estimates new pixels from smooth curves through 16 neighbours, so enlarged letters stay smooth instead of blocky.
Between 1000 and 2000	Leave as is	Already a good size

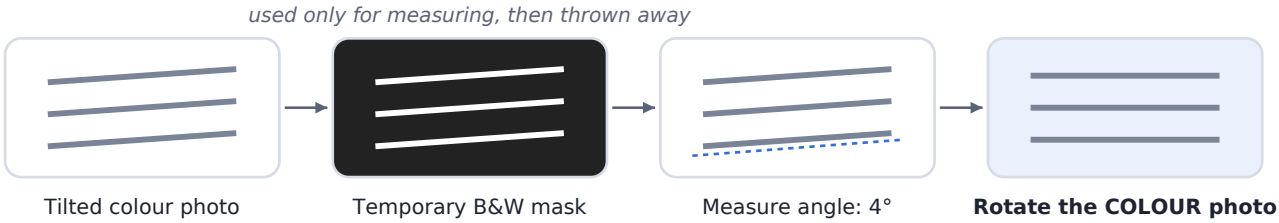
2.3 Deskew — Otsu thresholding + minAreaRect

Goal: measure how many degrees the text is tilted, and rotate it straight. Rows in step 3 assume horizontal text: at 5° tilt, the two ends of a 1000-pixel line differ by ~87 pixels in height — enough to split one bill line into two rows.

Thresholding = choose a brightness cut-off and split pixels into darker (ink) and lighter (paper). **Otsu's method** chooses the cut-off automatically: it builds a histogram (how many pixels have each brightness), tries every cut-off 0–255, and picks the one where both groups are tightest — the valley between the ink hill and the paper hill.



- Grey copy → Otsu → temporary mask where ink = white (THRESH_BINARY_INV).
- Collect the coordinates of all ink pixels (need ≥ 10, otherwise angle 0).
- `cv2.minAreaRect`: tightest rotated rectangle around them; its angle = text tilt. OpenCV ≥ 4.5 gives 0–90°, so angles above 45 have 90 subtracted (→ −45 to 45).
- If 0.5° < |angle| ≤ 15°: `getRotationMatrix2D` + `warpAffine` rotate the **colour** photo (cubic smoothing, new corners filled white like paper).
- The mask is thrown away — it was only a measuring tool.



2.4 Binarization — and why the OCR never gets a black-and-white image

Binarization = turning an image into only two values, pure black and pure white, using a cut-off. **Hard binarization** = doing that to the image the OCR reads, so grey detail is lost for good.



The faded ink (140) is lighter than the cut-off, so it turns WHITE and disappears. A faded digit or decimal point is lost for ever.

Faded thermal print and thin decimal points sit in the grey middle. Binarize, and they can vanish — a lost decimal point turns 190.00 into 190000. PaddleOCR is a neural network trained on normal colour photos, so it reads grey text better from the full image. So: Otsu is used to **measure** (like a ruler), and the OCR always reads the full photo. Both statements are true at once.

2.5 Quality check

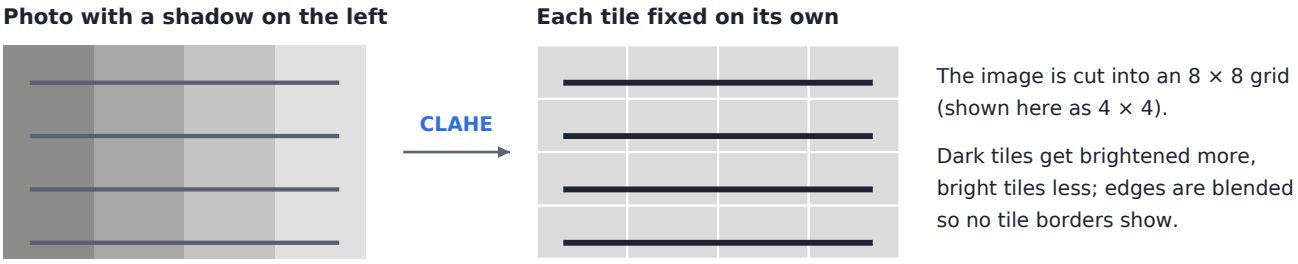
Measure	How it is computed	Meaning	Threshold
Blur	Variance of the Laplacian of the grey image	The Laplacian lights up edges. Sharp text = many strong edges = high variance; blurry = low	< 60 → LOW_QUALITY
Brightness	Mean grey value (0–255)	Low = dark photo	< 60 → LOW_QUALITY
Contrast	Standard deviation of grey values	Low = washed-out photo	Recorded only

The tag is only a note: the bill is still processed. 28 of 48 of your photos were tagged and still processed, because the maths check catches misreads anyway. No upper brightness limit: bright paper is normal.

2.6 Enhancement — CLAHE + denoise (makes "enh")

CLAHE = Contrast Limited Adaptive Histogram Equalization, word by word:

- **Histogram equalization:** stretch brightness values over the full 0–255 range so dark things get darker and light things lighter — text stands out.
- **Adaptive:** do it per tile (8 × 8 = 64 tiles), so a shadowed corner gets a strong boost and a bright area a small one. Tile edges are blended so no grid shows.
- **Contrast Limited:** each tile's boost is capped (clipLimit 2.0); otherwise plain paper grain would be stretched into specks that look like dots or commas.
- **Lightness only:** convert to LAB colour (L = lightness, A/B = colour), apply CLAHE to L, convert back — colours do not shift.



One brightness knob for the whole photo cannot fix both sides; 64 small knobs can.

Denoise (fastNlMeansDenoisingColored, strength 3): for each 7 × 7 patch, find similar patches in a 21 × 21 area and average them — random grain cancels, repeated shapes (letter strokes) survive. Kept light so decimal points are not erased.

2.7 Does preprocessing help? (measured)

Set	orig alone	enh alone	merged	Merged beats both
CORD held-out (50)	0.927	0.925	0.935	41/50 (82%)
Your desktop photos (48)	0.961	0.958	0.967	43/48 (90%)
Synthetic clean (60)	0.980	0.977	0.982	55/60 (92%)

Mean OCR confidence on number-containing lines. enh alone is slightly worse; merging both is best on every set. Confidence is not accuracy, and the effect on final field accuracy was not measured.

Part 3 — Technical architecture

Every component in the project. **Meaning** = what the thing is in general. **Role here** = exactly what this project does with it. **Why chosen** = the reason, compared with the alternatives.

Component	Meaning	Role in this project (in depth)	Why chosen	Alternatives (and why not)	Limitations
Python 3.10	Programming language	All code: 8 modules in src/, 15 scripts, 139 tests, the app	Every OCR / vision / ML library is Python-first; Decimal built in	Java / C++ (slower to build, fewer OCR tools)	Slower than C++ for pure loops (rules are fast anyway)
NumPy 1.26.4	Library for grids of numbers (arrays)	Holds every image as a height × width × 3 array passed between Pillow, OpenCV and PaddleOCR	Standard format all three accept	—	Pinned below 2.0 for PaddlePaddle 2.6 compatibility
Pillow 12.3.0	Simple image library	`Image.open` + `ImageOps.exif_transpose` to load and turn photos upright; draws the synthetic test bills	Best, simplest EXIF handling; good text drawing	OpenCV imread (ignores EXIF in some versions)	Not for heavy processing
OpenCV 4.6 (cv2)	Large, fast computer-vision library	Colour conversions, resize, Otsu threshold, minAreaRect, rotation, Laplacian blur check, CLAHE, denoise, drawing OCR boxes, saving JPGs	Fast (C++ inside), has every classical function needed	scikit-image (slower, fewer functions)	BGR colour order must be handled
PaddlePaddle 2.6.2	Baidu's deep-learning engine (like TensorFlow / PyTorch)	Runs PaddleOCR's neural networks on the CPU; never called directly	Required by PaddleOCR	—	Large install
PaddleOCR 2.7.3 (PP-OCRv4)	Pretrained OCR toolkit: 3 neural networks	Detector (DB) finds text lines; direction classifier flips upside-down ones; recogniser (SVTR) reads characters + confidence. English, CPU, created once, called twice per bill	Free, offline, strong on printed receipts, works on CPU, no training needed	Tesseract (weaker on phone photos, wants binarization); EasyOCR (slower on CPU); Google Vision / Textract (paid, online, private data leaves the PC)	Misreads digits; about 9 s per bill for both passes on CPU

Component	Meaning	Role in this project (in depth)	Why chosen	Alternatives (and why not)	Limitations
Two-pass IoU merge	Combining two OCR readings by box overlap	orig + enh read; boxes with IoU > 0.5 paired (keep higher confidence); enh-only boxes with IoU < 0.3 added	Raised confidence on every set	Single pass (cheaper); many variants + voting (slower)	Doubles OCR time
Row builder (own code)	Groups boxes into bill lines and columns	Vertical-overlap grouping ≥ 0.5; new cell at gap > 2.5 characters	OCR gives loose boxes; bills are tables; needs no training	LayoutLM / table-detection networks (need training data)	Curved or very tilted bills can merge/split rows
RapidFuzz 3.14.5	Fast fuzzy text matching (0–100 similarity)	Labels rows (21 labels, score ≥ 85); month names; merchant scoring in evaluation (≥ 80)	Tolerates OCR spelling slips; very fast	Exact match (misses "TOTA1"); fuzzywuzzy (same scores, slower)	Only knows the keywords in its list
Regular expressions (re)	Text patterns	Money, 5 date formats, GSTIN, bill no., tax/rate lines	Precise and fast for fixed formats	Hand-written character loops	Each new format needs a pattern
Decimal (Python)	Exact base-10 numbers	Every amount, sum and comparison	Float cannot store 0.1 exactly; errors would break exact balance checks	float (inexact)	Slightly slower (irrelevant here)
Field extractor (own code)	Candidate scoring	Several candidates per field; points for clues (label, position, arithmetic agreement); top 3 kept	Explainable; works on unseen layouts with zero training	LayoutLMv3 / Donut (need many labelled bills, can't explain)	Merchant name weak (39–73%)
Validator (own code)	Arithmetic proof + evidence-based repair	Two tax modes; tolerance max(0.05, 0.1%); witnesses; unique-fix repairs; 4 statuses; confidence	Turns maths into proof; never invents values	Trust OCR blindly; ML confidence threshold	Cannot tell fraud from misread; consistent forgery passes
Anomaly rules (own code)	Weighted warnings	7 flags → risk 0–1 (used by the app)	Each flag has a clear reason	Pure ML detector (no labelled fraud data)	Weights set by hand
scikit-learn 1.5.2 Isolation Forest	Unsupervised outlier detector	Experiment only (run_synthetic.py): fitted on 30 clean fake bills, 3 features	Needs no fraud examples	One-class SVM, autoencoder	Added nothing; not used by the app

Component	Meaning	Role in this project (in depth)	Why chosen	Alternatives (and why not)	Limitations
Streamlit 1.60	Turns a Python script into a web page	Upload/pick bill → full pipeline → status, overlay, cards, tables, JSON download; caches the OCR engine	A real UI in pure Python, shares the exact src/ code	Flask + HTML (more work), Gradio	Single user, demo only
JSON	Named-field text format	Per-stage files and final bill_result.json	Readable by people and programs	CSV (no nesting)	—
datasets 5.1 (Hugging Face)	Dataset downloader	Fetches the first 150 CORD-v2 receipts	Official source of CORD	Manual download	Needs internet once
pytest 9.1.1	Test runner	139 unit tests on made-up inputs (137 pass)	Standard	unittest	2 known failing edge cases
matplotlib	Chart library	5 report figures (make_report_assets.py)	Standard	—	Missing from requirements.txt

3.1 Deep dive — PaddleOCR (where the AI is)

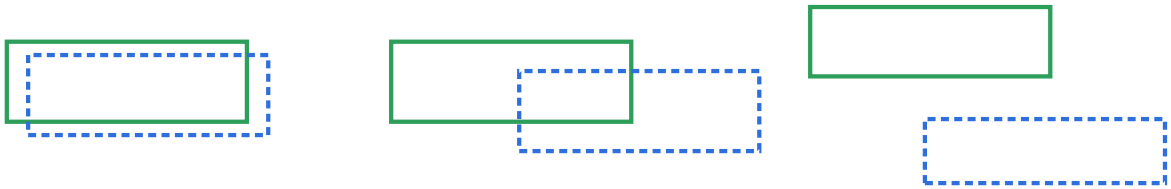
Network	Job	How, in simple words
1. Detector (DB = Differentiable Binarization)	Find where text is	Looks at every pixel and predicts "is this part of a letter?" as a probability map. Groups the yes-pixels into one box per text line. (This learned "binarization" is inside the model and is not the same as binarizing the photo.)
2. Direction classifier	Upright or upside down?	For each box, decides 0° or 180° and flips it if needed (use_angle_cls=True).
3. Recogniser (SVTR)	Read the characters	Looks along the cropped line left to right and outputs characters plus a confidence 0–1.

Created once: `PaddleOCR(use_angle_cls=True, lang="en", use_gpu=False)` — loading takes seconds, so it is reused for every bill. Each call returns e.g. box [[102,840],[410,840],[410,872],[102,872]], text "Grand Total 378.00", confidence 0.97. PaddleOCR only reads; understanding the bill is the project's own code.

3.2 Deep dive — IoU and the two-pass merge

IoU = area where two boxes overlap ÷ area both cover together (1 = same place, 0 = apart).

Green = box from original photo Blue dashed = box from enhanced copy



Big overlap (IoU > 0.5)
same text → keep the more confident

Partial overlap (0.3–0.5)
unclear → dropped

Tiny overlap (IoU < 0.3)
new text → added

3.3 Deep dive — fuzzy matching (RapidFuzz)

What it compares: the words of each OCR line against 21 label keyword lists (GRAND_TOTAL: "GRAND TOTAL", "NET AMOUNT"…; TOTAL; SUBTOTAL; CGST; SGST; CASH; CHANGE; DISCOUNT; ROUND OFF; DATE; Indonesian JUMLAH, PPN…). **How:** count the fewest single-letter deletions/insertions to turn one into the other; score = 100 × (1 − edits ÷ total letters).

Compared	Score	Result at cut-off 85
GRAND TOTA1 vs GRAND TOTAL	90.9	Match — OCR slip tolerated
CHANGE vs CHARGE	83.3	No match — service CHARGE is not mistaken for CHANGE
T0TAL vs TOTAL	80 → 100	Letter fix first (0→O in words with 3+ letters), then match
CASH vs CESS	50	No match
CGST vs SGST	75 (partial 85.7)	No match — partial matching is only used for keywords of 5+ letters, exactly to stop this

Partial matching finds a keyword inside a longer line ("GRAND TOTAL INR :" contains "GRAND TOTAL" → 100). Note TOTAL also partially matches inside SUBTOTAL (100), so a Sub Total line can carry both labels; the total scoring in step 5 then gives SUBTOTAL −2 and keeps them apart.

3.4 Deep dive — the arithmetic rules (validator)

#	Rule	Example
1	**Bill must add up** — tax on top: items + tax + service − discount ± round-off = total; tax inside: items − discount ± round-off = total. Both tried; smaller difference wins.	360 + 9 + 9 = 378
2	**Allowed difference** = max(0.05, 0.1% of total).	378 → 0.38; 2149 → 2.15
3	**qty × rate = amount** (within 0.05).	2 × 120 = 240
4	**CGST = SGST** (within 0.05).	9.00 = 9.00
5	**tax = base × rate** on rate lines (within 0.10).	360 × 2.5% = 9.00
6	**Real GST rates only:** 0, 0.25, 3, 5, 12, 18, 28% (halves for CGST/SGST); VAT/CESS ≤ 30%.	CGST 9% ok; 7% no
7	**Tax ≤ 35% of items.**	Stops a misread total being used as tax
8	**Round-off ≤ 1.00.**	499.60 → 500
9	**Witnesses** (independent proof of the total): TOTAL label, cash − change, GST table total, taxable + tax, subtotal chain, TOTAL GST = CGST + SGST.	400 − 22 = 378
10	**Repair only with proof:** exactly one fix (qty × rate, or one confusable digit: 0↔8, 1↔7, 5↔6, 3→8…) must balance the bill, the total must have a witness, and the fix is logged. Two possible fixes → no fix.	Item 170 read for 110

3.5 Deep dive — Isolation Forest (purpose, why, where, when)

- **Purpose:** an experiment — "can machine learning spot tampered bills better than my 7 rules?"
- **What it is:** 200 random trees, each cutting the data at random places. Normal bills sit in a crowd and need many cuts to isolate; unusual ones are isolated in few cuts → high "unusual" score (0–1).
- **Why this method:** it needs no fraud examples (unsupervised) — none with known answers existed; it could catch odd combinations no rule describes; small and fast.
- **What it looks at:** 3 numbers per bill: how far from balancing; lowest OCR confidence of items and total; tax ÷ items.
- **Where:** functions in `src/anomaly.py` (`fit_iforest`, `iforest_score`), called **only** by `scripts/run_synthetic.py`. The app and the normal pipeline never use it.
- **When (order):** make 60 clean + 60 tampered fake bills → run steps 1–7 → fit on the 30 clean bills of pairs 0–29 → score all → pick the cut-off by best F1 on pairs 0–29 → test on pairs 30–59: rules vs rules + forest.
- **Result:** identical (30/30 caught, 7/30 false alarms). Its main clue (does it balance?) is already the rules' strongest flag, so it adds nothing → kept optional. A valid negative result.

Part 4 — Parameters: what they do and how to tune them

A parameter is a fixed number in the code that a person chose. "Raised" and "lowered" below describe what would happen — the mechanism. The project did not measure each one separately; they were set by hand while fixing errors on the development bills.

4.1 Preprocessing (src/preprocess.py)

Parameter (file)	Value	What it controls	If raised ↑	If lowered ↓	How to tune it
MAX_SIDE	2000 px	Largest allowed long side	More detail kept; OCR slower (time grows with pixels), more memory	Faster; small print (GSTIN, decimals) can blur	Try 1600 / 2000 / 2400; compare accuracy and seconds per bill
MIN_SIDE / UPSCALE_TO	1000 / 1200 px	When and how much small images are enlarged	More images enlarged, slower; enlarging adds no real detail	Tiny images stay too small to read	Look at small screenshots; raise only if their text is missed
DESKEW_MIN_DEG	0.5°	Smallest tilt that gets corrected	Bills tilted 1° stay tilted → long rows may split	Rotates for tiny noise; each rotation slightly softens the image	Check row splits on slightly tilted bills
DESKEW_MAX_DEG	15°	Largest tilt trusted	A logo or border can fool the angle → a straight bill rotated wrongly	Genuinely tilted 10–15° photos left tilted	Inspect 02_overlay.jpg of rotated bills
BLUR_MIN / BRIGHT_MIN	60 / 60	LOW_QUALITY tag	More bills tagged	Fewer tagged	Tag only — no effect on results; set so tagged photos really look poor
CLAHE_CLIP	2.0	Contrast boost limit	Faded text darker, but paper grain becomes specks (fake dots/commas)	Weaker effect (1.0 ≈ none)	Try 1.5 / 2.0 / 3.0; watch the merged confidence
CLAHE_TILE	8 × 8	Tile grid size	More tiles = more local, more blocky artefacts	Fewer tiles = closer to one global boost; shadows not fixed	Rarely changed
DENOISE_H	3	Denoise strength	Smoother, but thin strokes and decimal points vanish (8 → 0)	More grain kept	Keep low; raise only for very grainy photos
Denoise windows	7 / 21 px	Patch size / search area	Slower, slightly smoother	Faster, less effective	OpenCV's recommended defaults

4.2 OCR merge and rows (src/ocr.py, src/rows.py, src/keywords.py)

Parameter (file)	Value	What it controls	If raised ↑	If lowered ↓	How to tune it
MATCH_IOU	0.5	Overlap needed to call two boxes the same line	Slightly shifted boxes not paired → duplicates or lost better reading	Neighbouring lines wrongly paired → one line lost	0.5 is the standard "same object" value
NEW_BOX_IOU	0.3	Overlap below which an enh-only box is added	More enh boxes added → same text twice (double items)	Missed lines not recovered	Count duplicate rows vs lines missed on dev bills
ROW_OVERLAP	0.5	Vertical overlap to join a row	Tilted/curved bills split one line into two (name separated from price)	Two neighbouring lines merged into one	Check 03_rows.txt for split or merged rows
CELL_GAP_CHARS	2.5 chars	Gap that starts a new column	Name and quantity glued into one cell	Item names with spaces split into cells	Check item rows in 03_rows.txt
MIN_SCORE	85	Fuzzy score needed for a label	Misspelled labels missed (no TOTAL found)	False labels: at 80, "CHARGE" (83) is read as CHANGE	Must stay above 83.3 (CHANGE/CHARGE)
PARTIAL_MIN_LEN	5 letters	Shortest keyword allowed to match inside longer text	"TOTAL" not found inside long lines	CGST matches SGST (85.7) and TAX matches inside words	Keep at 5
LETTER_FIX	words with 3+ letters	Where 0→O, 1→L, 5→S, 8→B is applied	Fewer fixes	Numbers could be changed into letters	Keep: numbers must never change

4.3 Numbers, fields (src/normalize.py, src/extract.py)

Parameter (file)	Value	What it controls	If raised ↑	If lowered ↓	How to tune it
MAX_INT_DIGITS	6	Longest ungrouped integer accepted as money	Phone numbers, HSN codes taken as amounts	Real big amounts (1250000) rejected	Check rejected numbers on dev bills
MONTH_FUZZ_MIN	75	Similarity for month words	Misread months ("JAM") missed	Random words read as months	Check date misses
Year range	1995 – next year	Valid date years	Old false dates accepted	Old real bills rejected	Rarely changed
TOL	0.05	qty × rate = amount tolerance	Wrong qty/rate pairs accepted	3 × 33.33 = 99.99 vs 100 fails	Look at rounding on item lines
BASE_TOL	0.10	tax = base × rate tolerance	Wrong tax lines accepted (3.80 for 3.75)	Correctly rounded tax rejected	At 0.10, a printed 3.80 (true 3.75, off by 0.05) still passes — likely why failing test 2 fails
MAX_TAX_SHARE	35%	Maximum tax as share of items	Misread totals accepted as tax	Real 28% GST + cess bills rejected	Check high-tax bills (alcohol, tobacco)
MAX_OTHER_RATE	30%	Max VAT/CESS rate	Odd rates accepted	Real VAT rates rejected	Set to the highest legal rate in your data
STRONG_FOR_DECIMAL	3	Amounts with 2 decimals needed to call a bill "decimal style"	More bills treated as integer style	Integer bills ("Tea 20") misjudged	Rarely changed
MERCHANT_ROWS / BOX_TOP / BOX_MIN	8 / 25% / 3.0	Where and how strongly the shop name is searched	Deeper search, more noise (body text wins)	Names printed lower are missed; more fallbacks	Merchant accuracy on dev bills (currently weak)
Total label scores	+4, +3, +2, +1, −2, −3	Net / grand / total / amount / subtotal / cash	That label dominates	That label matters less	Change only with arithmetic evidence

4.4 Validation and warnings (src/validate.py, src/anomaly.py)

Parameter (file)	Value	What it controls	If raised ↑	If lowered ↓	How to tune it
Balance tolerance	max(0.05, 0.1%)	Difference still counted as "adds up"	More VALID, but small misreads pass (higher risk)	Correct bills with rounding become CHECK_REQUIRED	Most important: watch VALID precision first
SURE_CONF	0.95	Values read above this are repaired only if the bill then balances	Fewer values count as "sure" → more repairs allowed	More values protected → fewer repairs	Watch REPAIRED precision
MAX_QTY	99	Largest quantity tried in repairs	Slower; odd fixes possible	Bulk quantities not repairable	Rarely changed
DIGIT_POOL	3	How many low-confidence values are tried for a one-digit fix	More chances of two fixes (→ none) or a lucky wrong fix	Fewer repairs	Keep small
ROUNDOFF_MAX	1.00	Largest unprinted round-off accepted	Misreads hidden as "round-off"	Rupee rounding (up to 0.99) rejected	Keep at 1.00 for INR
MERCHANT_LOW	0.5	Shop-name confidence that triggers review	More bills flagged for review	Fewer	Set by reviewer workload
Confidence factors	0.85 / 0.7 / 0.4	Confidence per repair / unverified items / CHECK_REQUIRED	Display only	Display only	Does not change status
WEIGHTS	0.15 – 0.5	Risk contribution of each flag	That flag dominates risk	That flag matters less	Pick so the order matches what reviewers care about
Risk / IF cut-offs	chosen by F1	Detect or not (experiment)	Fewer detections and false alarms	More of both	Automatic: best F1 on calibration pairs 0–29
IF n_estimators / seed	200 / 11	Forest size / repeatability	Steadier, slower	Noisier	200 is plenty for 3 features

4.5 How to tune a parameter properly (step by step)

- **1. Use only the development set** (20 desktop dev bills, CORD 0000–0099). Never look at CORD 0100–0149 or the 12 fresh bills while tuning — they are your honest test.
- **2. Record a baseline:** total accuracy, VALID/REPAIRED precision, share of CHECK_REQUIRED, and test results (`python -m pytest -q`).
- **3. Change one parameter at a time**, to one or two nearby values (e.g. tolerance 0.05% / 0.1% / 0.2%).
- **4. Re-run only what is affected:** preprocessing or OCR parameters need steps 1–2 again (~9 s per bill: `run_stage12.py`); row/field/validation parameters only need `run_stage34.py`, `run_stage5.py`, `run_stage6.py` (seconds).
- **5. Compare** with the baseline. Priority: VALID/REPAIRED precision must not drop; then total accuracy; then fewer CHECK_REQUIRED.
- **6. Look at the bills that changed** in outputs/stages/ to understand why.
- **7. Keep the change** only if it helps and all tests pass. Evaluate on the held-out sets once, at the end.

> **Example: the trade-off in the balance tolerance**

Raise it from 0.1% to 0.5%: bills with real rounding gaps become VALID (fewer reviews), but a misread total that is 0.3% off also becomes VALID — a wrong number nobody checks. Lower it to 0.05%: almost nothing wrong passes, but correct bills with paise rounding go to review. The project chose 0.1% to keep VALID precision high (97% on unseen CORD).

Part 5 — All your questions, answered

Q1. "Phones store the rotation in hidden EXIF data" — what does it mean?

EXIF is hidden info inside a phone photo, including an Orientation number. The phone saves the picture as the sensor captured it (often sideways) plus a note like "Orientation = 6: turn 90° clockwise". Gallery apps obey the note; Python does not. So the first function, `load()`, reads the note and rotates the pixels upright before anything else. Without it the OCR gets a sideways bill and reads almost nothing. (Picture in Part 2.1.)

Q2. What exactly is two-pass OCR?

The bill is read twice by PaddleOCR: pass 1 on the straightened original (orig), pass 2 on the enhanced copy (enh). Each pass gives boxes + text + confidence. The merge pairs boxes in the same place (IoU > 0.5) and keeps the more confident reading; boxes only enh found (IoU < 0.3) are added. Example: orig "378.0O" (0.83) vs enh "378.00" (0.97) → enh kept. Measured: 0.927 / 0.925 / 0.935 (orig / enh / merged) on unseen CORD.

Q3. Why split the work into many small stages? (debuggable, replaceable, testable)

- **Debuggable:** each stage saves its own file, so for a wrong bill you can see where it went wrong: OCR misread (02_ocr.json), rows merged (03_rows.txt), wrong total chosen (04_fields.json), or maths (05_validation.json).
- **Replaceable:** stages only talk through their outputs, so you could swap PaddleOCR for another OCR tool without touching the maths checks.
- **Testable:** each stage can be tested alone with small made-up inputs — 139 tests in tests/, 137 pass.

Q4. "In the app: plain function calls in memory, nothing written to disk"?

In the web app, each step passes its result directly to the next inside the computer's memory, like passing a note hand to hand: `preprocess.run` (clean) → `ocr.two_pass` (read twice) → `rows.build_rows` → `extract.extract` → `validate.validate` → `anomaly.score`. In the test scripts, each step saves a file instead, so the slow OCR does not have to be repeated.

Q5. "Open with Pillow, apply the rotation tag, convert RGB to BGR"?

Code	Meaning
`Image.open(path)`	Pillow opens the file
`ImageOps.exif_transpose(...)`	Turn upright using the EXIF note
`.convert("RGB")`	Ensure a normal 3-colour image (some are grey or have transparency)
`np.asarray(...)`	Turn it into a grid of numbers
`cv2.cvtColor(..., RGB2BGR)`	Swap colour order for OpenCV

Pillow stores each pixel as Red, Green, Blue; OpenCV expects Blue, Green, Red. Without the swap a red pixel (200, 30, 30) would be treated as blue.

Q6. "Pillow's EXIF transpose rotates the pixels to match the note"?

`exif_transpose` is one Pillow function ("transpose" = turn or flip). It reads the Orientation number (1–8): 1 = already upright, 3 = turn 180°, 6 = turn 90° clockwise, 8 = turn 90° anti-clockwise (2, 4, 5, 7 are mirrored versions). It turns the pixels to match, then resets the note to 1 so nothing rotates the picture a second time.

Q7. Why the black-and-white Otsu mask — what is happening?

Only to measure the tilt. Otsu picks the ink/paper cut-off automatically (valley between the two histogram hills); dark pixels become "ink" in a temporary mask; minAreaRect fits the tightest rotated rectangle around all ink pixels; its angle is the tilt; the **colour** photo is rotated by that angle; the mask is thrown away.

Q8. Why is the mask never shown to the OCR? What is binarization?

Binarization = making an image pure black and white with a cut-off. Doing it to the image the OCR reads ("hard" binarization) erases faded grey digits and thin decimal points (190.00 → 190000). So the mask is like a ruler: used to measure the tilt, never hung on the wall. The OCR always reads the full photo. That is how the project uses Otsu and still obeys "no hard binarization".

Q9. OCR vs PaddleOCR — and how we use them

OCR is the task; PaddleOCR is the tool doing it (like "phone" vs "iPhone"; other tools: Tesseract, EasyOCR, Google Vision). PaddleOCR = 3 pretrained neural networks (detector, direction classifier, recogniser — Part 3.1), created once with English model on CPU, called twice per bill, returning box + text + confidence per line. After that, everything is the project's own code: merge, rows, numbers, fields, maths.

Q10. "Preprocessing, OpenCV + Pillow" — and the use of OpenCV

It is step 1. Pillow opens and turns the photo upright; OpenCV does the rest:

OpenCV function	Used for
`cvtColor`	RGB→BGR; colour→grey for measuring; colour→LAB for CLAHE
`resize`	Shrink to 2000 (INTER_AREA) / enlarge to 1200 (INTER_CUBIC)
`threshold` (OTSU)	Temporary ink mask for tilt
`minAreaRect`	Tilt angle
`getRotationMatrix2D` + `warpAffine`	Rotate straight, white corners
`Laplacian`	Blur check

OpenCV function	Used for
`createCLAHE`, `split/merge`	Contrast boost on lightness only
`fastNlMeansDenoisingColored`	Light grain removal
`polylines`	Draw OCR boxes on the overlay
`imencode` / `imdecode`	Save/load JPGs (works with any Windows folder name)

Q11. "Two-pass IoU merge (merge(), src/ocr.py)"?

The code that combines the two OCR readings using IoU (box overlap): > 0.5 → same line, keep the more confident; enh-only < 0.3 → missed line, add it; between → duplicate, drop it. Picture in Part 3.2.

Q12. Isolation Forest — use, why, where, when, purpose

Purpose: an experiment testing whether ML beats the rules at spotting tampering. It is unsupervised (needs no fraud examples), looks at 3 numbers per bill, and is fitted on 30 clean fake bills. It lives in `src/anomaly.py` but runs **only** in `scripts/run_synthetic.py`, after step 6 of each fake bill; the app never uses it. Result: no gain (30/30 caught, 7/30 false alarms, same as rules) — so it stays optional. Full detail in Part 3.5.

Q13. What are the arithmetic rules?

Ten rules (table in Part 3.4): the bill must add up in one of two tax modes; allowed difference max(0.05, 0.1%); qty × rate = amount; CGST = SGST; tax = base × rate; only real GST rates; tax ≤ 35% of items; round-off ≤ 1.00; witnesses prove the total; repairs only with a unique fix and a witness. Then: adds up → VALID; after a proven fix → REPAIRED; not → CHECK_REQUIRED; too little → INSUFFICIENT_EVIDENCE.

Q14. What is the use of RapidFuzz?

OCR misspells labels ("GRAND TOTA1", "T0TAL"), so exact matching would miss the total line. RapidFuzz scores how similar two texts are (0–100) by counting letter edits; a row gets a label at ≥ 85. Used for: labelling rows (21 labels), month names, and judging shop names in evaluation (≥ 80). Examples and why 85 in Part 3.3 and Part 4.2.

Q15. What is CLAHE and how does it work?

A contrast booster that works in 64 small tiles separately (so shadows and bright areas are both fixed), with a cap (2.0) so paper grain is not blown up into fake dots, applied to lightness only so colours stay. It produces the enhanced copy read in OCR pass 2. Picture in Part 2.6.

Q16. Pillow vs OpenCV — what is each?

Two image libraries. Pillow = simple, best at opening files correctly (EXIF) and drawing text; used to load and turn the photo upright and to draw the fake test bills. OpenCV = large, fast vision library; used for every processing step. Hand-over: Pillow → NumPy array → RGB-to-BGR → OpenCV.

Part 6 — Results and viva

Results that count (data never used while building)

Test set	Bills	Total correct	When it said VALID/REPAIRED, it was right
CORD receipts 100–149 (unseen)	50	39 of 45 (87%)	29 of 30 (97%)
Your 12 fresh bills	12	9 of 12 (75%)	7 of 8 (88%)
(Development, tuned on) CORD 0–99	100	85 of 96 (89%)	60 of 62 (97%)
(Development, tuned on) 20 desktop bills	20	15 of 20 (75%)	10 of 10 (100%)

Why the last column matters most: a wrong total marked VALID is never checked by a person; a CHECK_REQUIRED bill always is. Tampering test on fake bills: 30/30 caught, but 7/30 untouched bills also flagged — all one narrow "thermal" layout.

Short answers to likely questions

If asked…	Say…
Where is the AI?	PaddleOCR's neural networks read the bill; my rules check and explain the result using the bill's own maths.
Why not train a model?	Only 48 own bills; the pretrained reader is good; the maths check catches its mistakes.
Why no black and white?	It erases faint digits and decimal points; black and white is only a ruler for the tilt.
Can it detect fraud?	No — it detects numbers that do not add up; a consistent forgery passes.
Show a failure.	Fresh bill_17: 430.80 read instead of 430.00, but it still added up → VALID.
Biggest weaknesses?	Misread digits in totals, shop names, missing dates.
How fast?	~8–15 s per bill on a laptop CPU, mostly the two OCR passes.
Failing tests?	2 of 139: rate-line edge cases (cash equal to the taxable value; a wrong 3.80 tax accepted). Known, documented, fix pending.
What next?	Fix the 2 tests, better shop-name and date reading, more real test bills, compare with other OCR tools.

Confirm before the viva

1. Debugging bills: code excludes 01, 10, 37, 46, 47, 48; older notes say 01, 10, 25, 27, 37, 46, 48.

2. Fresh bills were labelled on 10 Oct but rules changed on 11 Oct — if those changes did not use the fresh bills, you can call them unseen.